PROGRAMACIÓN II · UNIDAD 2 · CLASE 17

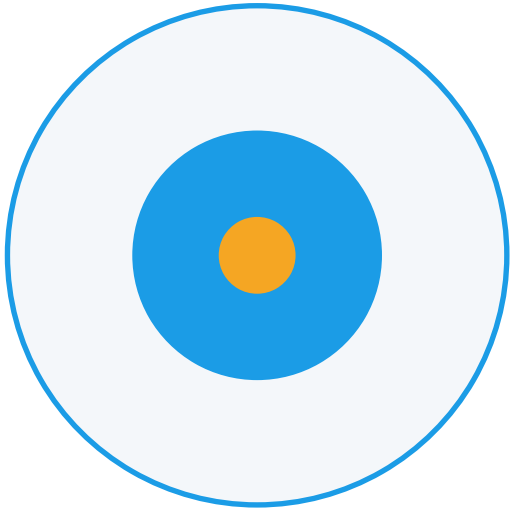
Principios S.O.L.I.D.

Introducción · SRP · OCP

> *“Cualquiera puede escribir código que una computadora pueda entender. Los buenos programadores escriben código que los humanos pueden entender.”*
>
> ~ Martin Fowler

Objetivo de la clase

Presentar los principios **SOLID**, su origen e importancia, y los dos primeros —**SRP** y **OCP**— como guías de diseño.



Qué veremos hoy

1. Qué son los principios SOLID
2. Por qué importan
3. SRP — Responsabilidad única
4. OCP — Abierto / Cerrado

1 ¿Qué son los principios SOLID?

Cinco postulados mencionados por primera vez a principios de los 2000 por **Robert C. Martin**; **Michael Feathers** los desarrolló y les dio el nombre de SOLID.

S	**Responsabilidad única**	`SRP`	hoy
O	**Abierto / Cerrado**	`OCP`	hoy
L	**Sustitución de Liskov**	`LSP`	
I	**Segregación de interfaces**	`ISP`	
D	**Inversión de dependencias**	`DIP`	

El resto (LSP, ISP y DIP) se ven en la próxima teórica.

Apunte Unidad 2, pág. 55

2 Por qué importan

- **Mantenibilidad** — Código más fácil de mantener: identificar y solucionar problemas, y sumar funciones sin efectos no deseados.
- **Escalabilidad** — Se adapta a los cambios y al crecimiento sin necesidad de grandes modificaciones.
- **Reusabilidad** — Conduce a componentes más reutilizables; ahorra tiempo y esfuerzo.
- **Colaboración** — Un marco común de trabajo que reduce los malentendidos en el equipo.
- **Reducción de errores** — Ayuda a identificar y mitigar errores comunes; software más robusto.

Apunte Unidad 2, pág. 56

3 SRP — Responsabilidad única

Una clase debe tener una sola responsabilidad → una única razón para cambiar (para ser modificada).

Beneficios

Pruebas

Una clase con una única responsabilidad tendrá (probablemente) menos casos de prueba.

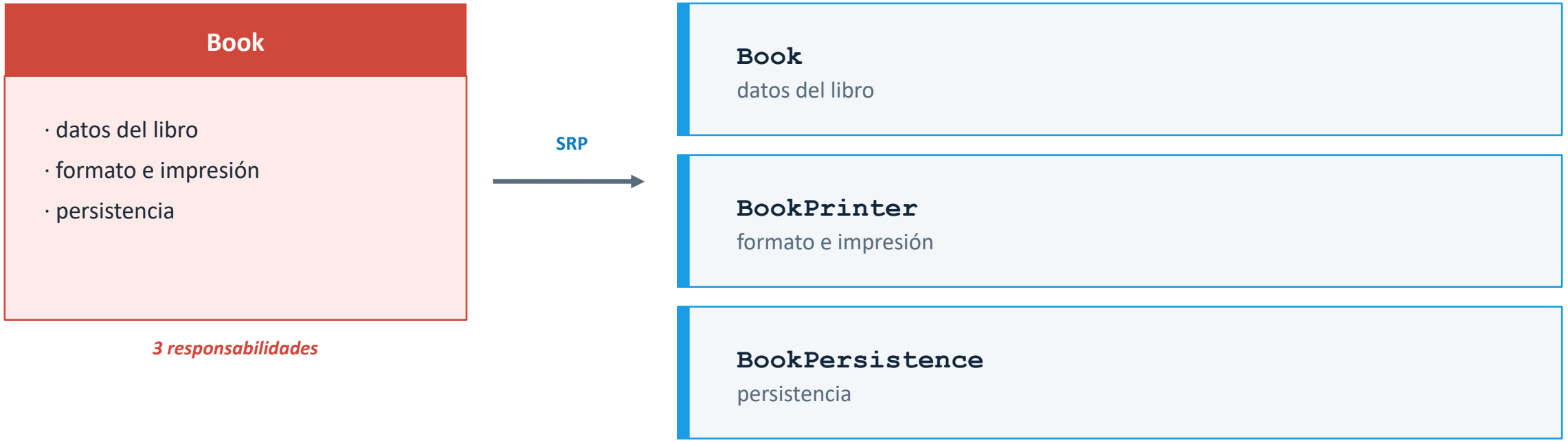
Menor acoplamiento

Menos funcionalidad en una sola clase implica menos dependencias hacia otras clases.

Apunte Unidad 2, pág. 56-57

3 SRP en acción: la clase Book

Book no sólo guarda los datos del libro: también los formatea, los imprime y los persiste. Demasiadas razones para cambiar.



Cada clase con una única responsabilidad.

Apunte Unidad 2, pág. 57-59

4 OCP — Abierto / Cerrado

Las clases deben estar abiertas a la extensión pero cerradas a la modificación: se agrega comportamiento nuevo sin tocar el código que ya funciona.

✓ **Abierto a la extensión**

Agregar comportamiento nuevo (por ejemplo, nuevas clases).

✗ **Cerrado a la modificación**

No cambiar el código existente que ya funciona.

La excepción a la regla: la corrección de errores (ahí sí se modifica el código).

Apunte Unidad 2, pág. 60-61

4 OCP en acción: IBookPersistence

IBookPersistence.cs

```
public interface IBookPersistence
{
    void Save(Book book);
}

public class TextFilePersistence : IBookPersistence
{
    public void Save(Book book) { /* archivo */ }
}

public class DbPersistence : IBookPersistence
{
    public void Save(Book book) { /* base de datos */ }
}
```

- Para agregar otra forma de guardar, se crea una clase nueva que implementa IBookPersistence.
- No se modifica el código existente: se extiende.
- El código cliente pasa a ser polimórfico e intercambiable.

Apunte Unidad 2, pág. 60-61

EN SÍNTESIS

- SOLID: 5 principios de diseño (Robert C. Martin; Michael Feathers acuñó el nombre).
- Importan por mantenibilidad, escalabilidad, reusabilidad, colaboración y menos errores.
- SRP: una clase, una responsabilidad → una única razón para cambiar (Book → Book/BookPrinter/BookPersistence).
- OCP: abierto a la extensión, cerrado a la modificación; la excepción es corregir errores.
- OCP con interfaces (IBookPersistence): agregar sin modificar; código cliente intercambiable.

Próxima clase (práctica): refactor de Book aplicando SRP y persistencia con interfaces (OCP).